

ISCTE – Instituto Universitário de Lisboa
Escola de Tecnologias Digitais Aplicadas

Análise de uma Rede Social de Programadores GitHub com Teoria de Grafos e Redes Complexas

Projeto da unidade curricular
Introdução a Redes Neurais

Docente: José Barros

Realizado por:

Rafaela Ferreira Nº 110826 – TDIA
Maria Costa Nº 130760 - TDIA
Rui Alves Nº 130755 – TDIA (desistiu)
Rafael Hacamo Nº 130736 – TDIA (desistiu)

Ano letivo 2025/2026
Sintra, 2026

Índice

1. Definição do problema	3
2. Revisão bibliográfica inicial.....	3
3. Identificação e tratamento dos dados.....	3
4. Configuração do sistema em rede.....	4
5. Questão de investigação.....	5
6. Metodologia geral.....	5
7. Tarefas da metodologia e métricas.....	5
8- Resultados e discussão	7
9- Integração com redes neuronais	11
Conclusões do trabalho final.....	12

1. Definição do problema

Este projeto estuda como se organiza e como funciona uma rede de influência entre programadores na plataforma GitHub. Pretende-se perceber se alguns programadores ocupam posições estruturalmente mais importantes na rede, se existem *hubs* e se há diferenças na estrutura de ligações entre programadores de desenvolvimento Web e de Machine Learning. Em termos simples, o problema pode ser formulado da seguinte forma: dado um conjunto de utilizadores ligados por relações de seguimento mútuo, como podemos representar esta rede como um grafo, identificar posições de maior importância e relacionar essas posições com o tipo de programador (Web ou ML).

Para isso, o projeto vai usar conceitos básicos de teoria de grafos e redes complexas, como nós, arestas e centralidade. A análise será feita sobre uma rede real de programadores GitHub, onde cada nó representa um utilizador e as ligações representam relações sociais na plataforma.

2. Revisão bibliográfica inicial

A teoria de grafos e redes complexas mostra que muitas redes sociais apresentam distribuições de grau assimétricas, com poucos *hubs* e muitos nós pouco ligados. Segundo Newman (2010), estas redes tendem a ter caminhos curtos e formação de triângulos (*small-world*), onde nós centrais têm maior capacidade de difundir informação ou ligar grupos. Medidas de centralidade, como grau ou intermediação, permitem identificar agentes mais influentes (Newman, 2010).

No contexto das redes sociais digitais, vários autores discutem como a posição de um indivíduo na rede se relaciona com o seu impacto sobre os outros, mostrando que estruturas de ligação moldam identidades, opiniões e comportamentos (Pompei, 2018). Em plataformas como o GitHub, seguir outro programador pode ser visto como um tipo de influência profissional, que mostra um reconhecimento de competência ou interesse em aprender com o seu trabalho.

Estudos sobre redes, como o de Rozemberczki et al. (2019), sugerem que existe uma relação profunda entre a estrutura das arestas e os *labels* dos nós. O conceito de homofilia, tendência de nós semelhantes se conectarem preferencialmente, é uma base essencial para este estudo. Ao analisar este dataset, pretendemos verificar, se os padrões de ligação são suficientemente distintos para diferenciar os grupos de Web Development e Machine Learning, focando apenas na morfologia do grafo (Rozemberczki et al., 2019).

3. Identificação e tratamento dos dados

Os dados usados neste trabalho provêm do conjunto “GitHub Social Network (musae-github)”, disponibilizado pelo Stanford Network Analysis Project (SNAP). Trata-se de uma grande rede social de programadores GitHub recolhida através da API pública em junho de 2019. Neste dataset, cada nó/vértice corresponde a um programador que colocou

“star” em pelo menos 10 repositórios. As arestas representam relações de “seguidores”, onde dois utilizadores se seguem mutuamente.

Neste projeto serão utilizados dois ficheiros principais para a análise prática. O ficheiro “musae_git_edges.csv” contém pares de IDs de utilizadores que se seguem mutuamente e será usado para montar a estrutura do grafo. O ficheiro “musae_git_target.csv” associa a cada ID o respetivo nome de utilizador e uma classe binária (desenvolvedor Web ou de Machine Learning). Estes rótulos serão usados como base para comparar estruturalmente os dois grupos de nós. Embora o dataset original também inclua o ficheiro “musae_git_features.json” com características técnicas, este trabalho focará a sua análise na topologia e conectividade da rede.

Para tratar os dados: verificámos se existiam valores nulos e se o tipo de dados estava correto, organizámos por IDs e confirmámos que não existiam arestas repetidas ou auto-ligações em “musae_git_edges.csv” e garantimos que todos os IDs das arestas em “musae_git_edges.csv” também existiam em “musae_git_target.csv”. Esta limpeza serve para garantir que o grafo utilizado é simples, consistente e adequado ao âmbito do projeto.

4. Configuração do sistema em rede

Do ponto de vista técnico, o sistema será representado por um grafo não dirigido $G = (V, E)$. O conjunto de vértices V é formado pelos IDs dos programadores presentes no dataset, onde cada vértice corresponde a um utilizador do GitHub. Esse utilizador tem associados dois atributos principais, provenientes do ficheiro “musae_git_target”, o seu nome de utilizador e um rótulo binário, que indica se o perfil corresponde a um programador de desenvolvimento Web ou de Machine Learning. O conjunto de arestas E corresponde às ligações de seguimento entre utilizadores. É construído a partir do ficheiro “musae_git_edges.csv”, que contém pares não orientado $\{u, v\}$, indicando que relação entre o utilizador u e o utilizador v é recíproca.

Neste trabalho, o grafo é não ponderado, ou seja, todas as ligações têm o mesmo peso, tal como explicitado no documento README.txt do dataset. As arestas são não orientadas e simples, o que implica que não serão consideradas múltiplas ligações entre o mesmo par de nós nem auto-ligações (loops).

A configuração do sistema em rede consiste assim em: construir a estrutura do grafo a partir do ficheiro de “musae_git_edges” e mapear para cada nó o respetivo rótulo social (Web ou ML) a partir do ficheiro “musae_git_target”. Esta configuração define de forma clara os elementos de base, vértices, arestas e rótulos, que serão usados nas fases seguintes do trabalho para analisar a topologia da rede e discutir questões de influência e posição relativa dos programadores no ecossistema do GitHub (Newman, 2010; Pralat et al., 2022). Ao ver a estrutura do grafo concluímos que este é totalmente conexo, tem 37700 vértices (utilizadores), 289003 arestas (ligações) e que, aproximadamente, 74% desses utilizadores pertencem a Web Development e 26% a Machine Learning.

5. Questão de investigação

“Em que medida a estrutura da rede GitHub revela padrões de homofilia e especialização entre developers Web e ML e como é que métricas de redes complexas podem ajudar a explicar e a melhorar a classificação binária destes developers?”

6. Metodologia geral

O objetivo geral do trabalho é caracterizar a estrutura da rede social GitHub e avaliar de que forma as métricas de redes complexas permitem identificar padrões de homofilia e especialização entre developers, Web e ML, e diferenciar a classificação binária dos nós entre estas duas categorias.

Para concretizar este objetivo geral, organizámos a metodologia em objetivos específicos que fomos cumprindo:

1. Descrevemos a rede em termos de dimensão, conectividade e distribuição dos rótulos Web/ML, garantindo a integridade estrutural do grafo utilizado na análise;
2. Caracterizamos a conectividade local dos nós através do grau e analisar se o tipo de developer (Web vs ML) influencia os padrões de ligação na rede;
3. Identificar nós estruturalmente centrais e “pontes” na rede utilizando métricas de centralidade (betweenness, closeness e PageRank), explorando diferenças entre developers Web e ML;
4. Medir a assortatividade de grau e de atributo (Web/ML), bem como o coeficiente de clustering, estudando se a rede apresenta correlações estruturais e padrões de homofilia entre developers do mesmo tipo;
5. Detetar comunidades na rede e analisar a sua composição (Web dominante, ML dominante, mista), relacionando-a com os padrões de homofilia e especialização.

7. Tarefas da metodologia e métricas

Para atingir estes objetivos depois de já ter limpado, organizado, criado o nosso grafo e sincronizado o target, foram-se aplicando as seguintes métricas e algoritmos, implementadas em Python num Jupyter Notebook, recorrendo às bibliotecas Pandas e NetworkX:

Estudo de Componentes: Para averiguar como poderíamos abordar o nosso estudo começámos por calcular o número de componentes conexas, com a função `number_connected_components()` do NetworkX, que apenas era uma.

Graus dos vértices e CCDF dos graus: Para caracterizar a conectividade local da rede, calculou-se o grau de cada nó e analisaram-se estatísticas descritivas para perceber o número médio de ligações por utilizador e a variabilidade desses valores. Em seguida, observou-se a distribuição dos graus no grafo completo e vimos de forma separada como estes valores mudavam por tipo de developer (Web vs ML). Por fim fizemos uma análise CCDF da distribuição de grau e visualizámos os gráficos. Assim conseguimos analisar se

o tipo de developer influencia o número de conexões e a forma como estas se distribuem na rede.

Centralidade de grau– Para consolidar o que já tinha sido visto calculámos a centralidade de grau de cada nó para perceber o que aqueles valores significavam em escala e o quão influentes eram os utilizadores mais influentes a nível global. Vimos as métricas a comparar os developers ML e Web e detetámos o top 10 mais central e onde pertenciam.

Centralidade de intermediação (betweenness centrality) – Utilizou-se o algoritmo padrão de betweenness em NetworkX para medir quantas vezes um nó aparece em caminhos entre pares de outros nós, e entender quais são as pontes da rede. Assim, analisam-se que perfis centrais que ligam sub-comunidades, para perceber padrões de ligação e áreas de interseção entre comunidades. Também calculámos o top 10 com maior central betweenness e pudemos perceber se estes utilizadores “ponte” são os mesmos que utilizadores de maior grau que já tínhamos encontrado.

Centralidade de proximidade (closeness centrality) – Para avaliar quão próximos se encontram os developers relativamente ao resto da rede, calculou-se a centralidade de proximidade para cada nó. Esta métrica foi incluída porque permite distinguir perfis estruturalmente “centrais” daqueles que se encontram mais periféricos, o que pode estar associado a diferentes formas de participação na comunidade. Comparámos novamente entre tipo de developer e vimos os utilizadores mais centrais segundo esta métrica.

PageRank – Aplicou-se ainda o algoritmo PageRank adaptado a grafos não dirigidos, para ter uma noção da importância relacional que não depende apenas do número de ligações, mas também da centralidade dos vizinhos. Com o PageRank podemos testar se developers Web ou ML tendem a concentrar mais “prestígio” na rede de seguidores mútuos, o que pode ter implicações na visibilidade e influência de cada grupo dentro do GitHub. Assim voltámos a ver as estatísticas comparatórias entre utilizadores Web e ML e ver quem são os mais influentes.

Assortatividade de grau e de atributo– Calculámos a assortatividade de grau, para medir se utilizadores com muitas conexões tendem a ligar-se a outros de elevado grau (assortatividade positiva) ou aos de grau baixo (dissortatividade) e vice-versa. A seguir calculámos a assortatividade de atributo, tendo como atributo o rótulo binário Web/ML, para avaliar quantitativamente a homofilia: valores positivos indicam que os utilizadores tendem a ligar-se a outros do mesmo tipo e baixos o contrário.

Clustering: Cálculo do coeficiente de clustering (global e médio por grupo Web e ML) para avaliar o grau de formação de triângulos e subestruturas coesas, verificando se a rede apresenta uma estrutura mais localmente densa em torno de determinados tipos de developers. Isto complementa a análise de homofilia ao mostrar se os laços entre developers do mesmo tipo estão organizados em triângulos e subestruturas coesas.

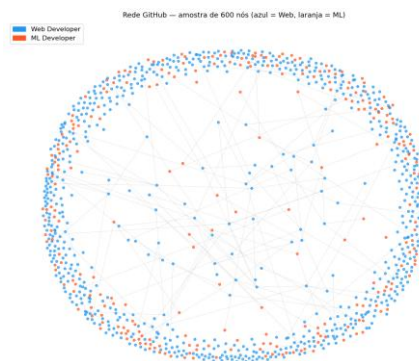
Deteção de comunidades – Depois detetámos comunidades, primeiro através do algoritmo Girvan-Newman, mas como este é muito pesado e fragmentador e não estava a conseguir detetar comunidades utilizámos o algoritmo Louvain. Esta etapa é crucial para estudar a homofilia, pois permite verificar se as comunidades estruturais são maioritariamente compostas por nós da mesma classe (Web ou ML) e até que ponto os limites entre comunidades acompanham a fronteira entre tipos de especialização.

Comparação SmallWorld – Para verificar formalmente se a rede GitHub apresenta a propriedade small-world, comparamos os valores calculados de clustering médio e comprimento médio do caminho com os valores esperados para um grafo aleatório Erdős-Rényi de referência com o mesmo número de nós e a mesma densidade.

8- Resultados e discussão

A análise confirmou que a rede GitHub estudada é um grafo grande e conexo, com 37 700 nós, 289 003 arestas e uma única componente conexas, o que permite interpretar as métricas num mesmo sistema estrutural. A distribuição dos rótulos também não é equilibrada, sendo cerca de 74% dos nós classificados como Web e 26% como Machine Learning, pelo que qualquer comparação entre grupos deve ter em conta essa assimetria de base.

Além disso, a densidade do grafo é muito baixa, cerca de 0,0004067, mostrando que a rede é extremamente dispersa e que cada programador está ligado apenas a uma pequena fração do total de utilizadores possíveis. Este resultado é típico de redes sociais grandes e ajuda a enquadrar as restantes métricas: trata-se de uma rede com baixa densidade global, mas com estrutura interna suficientemente rica para revelar hubs, comunidades e padrões de homofilia.



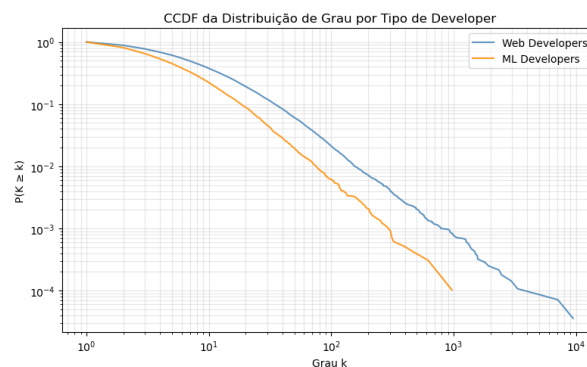
Componentes e conectividade

O estudo das componentes mostrou que a rede tem apenas uma componente conexas e que essa componente inclui a totalidade dos 37 700 nós. Isto significa que todos os utilizadores estão ligados direta ou indiretamente, não existindo sub-redes isoladas que impedissem a leitura global da estrutura. Isto sugere que o GitHub é um ecossistema fortemente integrado, ainda que internamente diferenciado. Em termos analíticos, permite

usar com confiança métricas como closeness, betweenness e análise de comunidades, porque todas são calculadas sobre uma rede estruturalmente unificada.

Grau dos vértices e distribuição dos graus

A análise do grau mostrou que a rede apresenta conectividade muito desigual, com muitos nós pouco ligados e poucos nós muito conectados, o que é consistente com a presença de hubs. A observação da distribuição e da CCDF dos graus reforça a ideia de uma rede heterogénea, em que uma parte pequena dos utilizadores concentra uma fração relevante das ligações e confirmou também a dominância dos hubs web, vistos mais tarde nas métricas de centralidade.



Na comparação entre grupos, os developers Web apresentam grau mediano superior ao dos developers ML, com mediana 6 para Web e 4 para ML, correspondendo a um rácio de 1,50. Este resultado indica que os utilizadores Web mantêm mais ligações diretas do que os de Machine Learning, o que os coloca em posições localmente mais conectadas dentro da rede.

Métricas de centralidade

Foram calculadas quatro métricas de centralidade: centralidade de grau, closeness centrality, betweenness centrality e PageRank. Em conjunto, estas medidas permitem analisar popularidade local, proximidade global, papel de ponte entre zonas da rede e importância relacional ponderada pela estrutura das ligações.

Os resultados mostram uma vantagem consistente dos developers Web em todas estas métricas. No resumo por medianas do notebook, os valores são superiores para Web em todas as features consideradas:

Métrica	Mediana Web	Mediana ML	Rácio W/ML
degree	6.000000	4.000000	1.50x
closeness	0.318651	0.290831	1.10x
betweenness	0.000001	0.000000	3.80x
pagerank	0.000014	0.000011	1.25x
clustering	0.109677	0.054545	2.01x

Isto mostra que a diferença entre os grupos não aparece numa só métrica isolada, mas de forma sistemática em vários níveis da estrutura da rede.

Outro exemplo disto é a diferença entre a média dos developers Web é 0,000071, enquanto a dos ML é 0,000027 quanto à central betweenness notando-se que o valor máximo Web chega a 0,251016, contra apenas 0,008302 em ML. Isto sugere que os principais nós-ponte da rede são claramente Web. Também no PageRank a diferença é clara: a média é 0,000029 para Web e 0,000018 para ML, e o máximo no grupo Web atinge 0,016514, enquanto em ML fica em 0,001420. Isto reforça a ideia de que os developers Web não só têm mais conexões, mas ocupam também posições de maior prestígio global na rede.

Ao observar o top 10 em cada centralidade mostra ainda notámos uma forte repetição dos mesmos nomes, como dalinhuang99, nfultz, addyosmani, Bunlong, gabrielponceicao, rfthusn e nelsonic, todos classificados como Web nos tops apresentados. Esta concordância entre métricas indica que alguns nós Web acumulam vários tipos de centralidade ao mesmo tempo: são hubs locais, pontes estruturais, nós próximos do resto da rede e também influenciadores globais.

Assortatividade e homofilia

A assortatividade de grau obtida foi -0,075217, um valor negativo mas próximo de zero. Isto indica uma tendência desassortativa fraca, quase 0. Ou seja, nós muito conectados quase não mostram preferência em ligar-se a nós menos conectados ou mais conectados ou vice versa. Já a assortatividade por atributo, usando o rótulo Web/ML, foi 0,377822, um valor positivo e relativamente elevado. Este resultado é muito importante para o projeto porque mostra homofilia moderada: developers Web tendem a ligar-se mais a Web e developers ML tendem a ligar-se mais a ML, o que significa que o tipo de utilizador não está distribuído aleatoriamente na rede.

Clustering e coesão local

O cálculo do coeficiente de clustering permitiu avaliar a formação de triângulos e o grau de coesão local da rede. Esta métrica é útil porque mostra se os vizinhos de um dado utilizador tendem também a estar ligados entre si. O clustering médio da rede é 0,167537, o que significa que, em média, cerca de 16,75% dos pares de vizinhos de um developer também estão ligados entre si. Já a transitividade global é 0,012357, o que significa que apenas cerca de 1,24% dos trios possíveis estão fechados em triângulos. Isto significa que existe alguma coesão local, com pequenos grupos de vizinhos ligados entre si, mas a coesão global em triângulos é baixa, o que é consistente com uma rede esparsa dominada por alguns hubs fortes.

A comparação por grupo mostra novamente uma ligeira vantagem dos developers Web: clustering médio de 0,171057 em Web contra 0,157432 em ML, e mediana de 0,109677 em Web contra 0,054545 em ML. A diferença aqui é menos marcada do que nas centralidades, o que sugere que a principal distinção entre os grupos está mais na posição estrutural global do que apenas na formação de círculos locais.

Deteção de comunidades

A deteção de comunidades foi um dos pontos mais importantes da análise mesoscópica da rede. Embora tenha sido inicialmente considerada a abordagem de Girvan-Newman, o algoritmo revelou-se demasiado pesado e fragmentador. Mesmo numa amostra de 1000 nós, gerou centenas de comunidades muito pequenas, chegando a 873 na primeira divisão, o que mostra fragmentação excessiva e pouco valor interpretativo para uma rede desta dimensão. Portanto, em alternativa, foi adotado o algoritmo de Louvain, mais apropriado para redes grandes.

Com Louvain, foram detetadas 34 comunidades e a modularidade foi 0,448518, valor que indica uma partição com estrutura significativa, com maior densidade de ligações dentro das comunidades do que entre elas.

As comunidades apresentam tamanhos muito heterogéneos: média de 1108,8 nós, mediana de 10,5 e máximo de 8983. Isto mostra que coexistem muitas comunidades pequenas com algumas comunidades muito grandes, revelando uma organização interna desigual mas estruturalmente relevante. A composição das comunidades confirma a homofilia observada antes. A pureza média é 0,838036, a mediana é 0,815621 e o máximo atinge 1,0, o que indica que, em geral, as comunidades são bastante homogéneas em termos de rótulo.

Entre as maiores comunidades, a comunidade 22 tem 8983 nós e é 94,1% Web, enquanto a comunidade 1 tem 8281 nós e é 69,6% ML. Isto mostra que o grupo ML, embora minoritário na rede global, forma pelo menos um núcleo estrutural de grande dimensão e coeso. De resto, em geral, várias comunidades grandes são fortemente dominadas por Web.

Na classificação final das 34 comunidades separámo-las em grupos tendo em conta a sua composição e os pesos dos targets na globalidade da rede e acabámos com: 18 Web comunidades dominantes, 10 mistas e 6 ML dominantes. Assim, existe segmentação estrutural relevante entre os grupos, mas não separação absoluta, já que quase um terço das comunidades continua a ser mista.

A existência simultânea de comunidades dominadas por um dos grupos e de comunidades mistas sugere um resultado equilibrado: há sinais de especialização e homofilia, mas também zonas de interseção entre perfis. Em termos analíticos, isto significa que a rede GitHub combina segmentação parcial com integração global. Este resultado é importante para a questão de investigação porque mostra que a classificação Web/ML tem correspondência estrutural, mas essa correspondência não é absoluta.

Small-world e leitura global

A verificação da propriedade small-world foi feita comparando a rede GitHub com um grafo aleatório de referência do tipo Erdős-Rényi, usando o mesmo número de nós e a mesma densidade. Esta comparação é importante porque permite perceber se a rede real

tem apenas ligações aleatórias ou se apresenta uma organização mais típica de redes sociais complexas, com vizinhanças muito coesas e caminhos curtos entre nós.

Os resultados mostraram que rede GitHub apresenta uma clara propriedade small-world. Embora o clustering médio já indicasse alguma coesão local, a comparação com um grafo de Erdős-Rényi mostrou que esse clustering é 412 vezes superior ao esperado numa rede aleatória equivalente, enquanto o comprimento médio dos caminhos se mantém praticamente idêntico ao de referência, com rácio 0,85. Isto significa que a rede combina grupos locais densos com distâncias globais curtas, o que é típico de redes sociais complexas e reforça a ideia de uma estrutura bem organizada e eficiente.

No conjunto, os resultados mostram que a rede GitHub não é aleatória. A topologia contém sinal estrutural claro sobre influência, proximidade, agrupamento local e especialização, respondendo de forma positiva à questão de investigação: existem padrões de homofilia e organização diferenciada entre developers Web e ML, ainda que com zonas de mistura entre os grupos.

9- Integração com redes neuronais

Os resultados mostram que a topologia da rede já contém informação útil para distinguir os dois tipos de developer. O modelo mais adequado seria uma Graph Neural Network, como uma Graph Convolutional Network ou uma Graph Attention Network, uma vez que estas arquiteturas são capazes de aprender diretamente a partir da estrutura do grafo e dos atributos dos nós.

Uma aplicação natural seria a classificação binária dos utilizadores em Web ou ML. Isto foi o que foi visto ao longo do trabalho, ao mostrar que métricas independentes calculadas apenas a partir da rede convergem na mesma direção e podem servir como features para um classificador, como um Multilayer Perceptron sobre métricas estruturais ou uma GCN que explore diretamente as ligações.

Para isso, seriam necessários dados adicionais, sobretudo atributos dos nós, como linguagens usadas, atividade em repositórios, número de seguidores ou as features do ficheiro `musae_git_features.json`. Esses dados permitiriam combinar informação topológica com informação técnica e melhorar o poder preditivo do modelo.

A principal diferença conceptual entre a abordagem atual e uma abordagem neuronal é a interpretabilidade. As métricas de grafos permitem explicar claramente o que significa grau, betweenness, clustering ou comunidades, enquanto uma GNN pode melhorar a classificação mas tende a ser menos transparente. Assim, a análise por grafos continua essencial como base interpretativa, e a rede neuronal surgiria como complemento preditivo.

Os principais desafios seriam o custo computacional, a dimensão do grafo, o desequilíbrio entre classes e a necessidade de definir bem treino, validação e teste.

Conclusões do trabalho final

Este trabalho mostrou que a rede social GitHub pode ser estudada de forma eficaz como um grafo não dirigido, no qual os nós representam programadores e as arestas relações mútuas de seguimento. A análise confirmou que a rede é grande, conexa, muito dispersa, mas ao mesmo tempo estruturalmente rica, com hubs, comunidades e padrões de homofilia relevantes.

Em relação à questão de investigação, os resultados indicam que a estrutura da rede revela diferenças reais entre developers Web e ML. Os developers Web apresentam, de forma sistemática, maior conectividade, maior centralidade, maior importância relacional e comunidades frequentemente dominantes, enquanto os developers ML surgem mais concentrados em alguns núcleos coesos, incluindo uma grande comunidade ML dominante.

Ao mesmo tempo, a existência de comunidades mistas e de uma única componente conexa mostra que a separação entre os grupos não é total. Assim, a rede combina homofilia e especialização com integração global, o que é uma conclusão mais rica do que uma simples divisão binária rígida entre Web e Machine Learning.

A principal conclusão metodológica é que as métricas de redes complexas conseguem descrever o sistema e também apoiar futuras tarefas de classificação dos nós. Como várias métricas independentes convergem na mesma distinção entre Web e ML, a topologia da rede codifica informação útil sobre o tipo de utilizadores, o que justifica a integração futura com modelos de aprendizagem automática sobre grafos.

Referências

- SNAP: Stanford Network Analysis Project. (2018, December 31). *GitHub social network* (musaegithub) [Data set] [<https://snap.stanford.edu/data/github-social.html>]
- Newman, M. E. J. (2010). *Networks: An introduction*. Oxford University Press. [[cs.cornell](https://cs.cornell.edu)]
- Pompei, M. (2018). Redes sociais: influência, identidade e diferença na sociedade contemporânea. *Em Sociedade*, 3(1), 45–60. [[periodicos.pucminas](https://periodicos.pucminas.br)]
- Rozemberczki, B., Allen, C., & Sarkar, R. (2019). *Multi-scale Attributed Node Embedding*. [[ArXiv:1909.13021](https://arxiv.org/abs/1909.13021).]
- Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Yu, P. S., & Philip, S. Y. (2020). *A comprehensive survey on graph neural networks*. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1), [<https://doi.org/10.1109/TNNLS.2020.2978386>]